

심화 실습 평가2 (리눅스 프로그래밍)

TCP 기반 원격 장치 제어 프로그램 — 프로젝트 제안서

항목	내용
과정	한화비전 VEDA
이름	방준한
제출일	2026년 6월 5일
서버	Raspberry Pi 4 (Daemon)
클라이언트	Ubuntu Linux

1. 프로젝트 개요

1.1 목적 및 배경

본 프로젝트는 TCP 소켓 통신을 기반으로, 원격지의 클라이언트(Ubuntu Linux)에서 라즈베리파이4 서버에 연결된 물리 장치(LED·부저·조도센서·7세그먼트)를 제어하는 시스템을 구현하는 것을 목표로 한다.

서버는 상시 동작하는 **데몬(Daemon) 프로세스**로 구성되어 네트워크 요청을 대기하며, 모든 장치 제어 로직은 **단일 공유 라이브러리(libcontrol.so)** 형태로 구성하여 기능 업그레이드 시 서버 재작성 없이 라이브러리 파일 교체만으로 반영할 수 있도록 설계한다. 이를 통해 유지보수성과 확장성을 확보한다.

클라이언트는 Ubuntu Linux에서 동작하는 **C 프로그램**으로, **계층적 메뉴 기반 터미널 클라이언트**다. RPi4 서버에 TCP로 접속하여 메뉴에서 선택한 명령을 전송하고 서버 응답을 출력한다. 응답은 **HTTP-like 텍스트 형식**을 사용하여, 추후 웹 클라이언트로 확장할 수 있도록 한다. 클라이언트는 실행 중 비정상 종료를 방지하기 위해 **SIGINT 신호에서만 종료**되도록 시그널을 처리한다.

1.2 주요 기능 요약

장치	기능
LED	클라이언트에서 on/off 및 밝기 3단계(최대 / 중간 / 최저) 제어 (소프트웨어 PWM)
부저(Buzzer)	클라이언트에서 (음악)소리 on/off 제어
조도센서	I2C 아날로그 CDS (0x48) 원시값 읽기; 임계값 판단은 서버

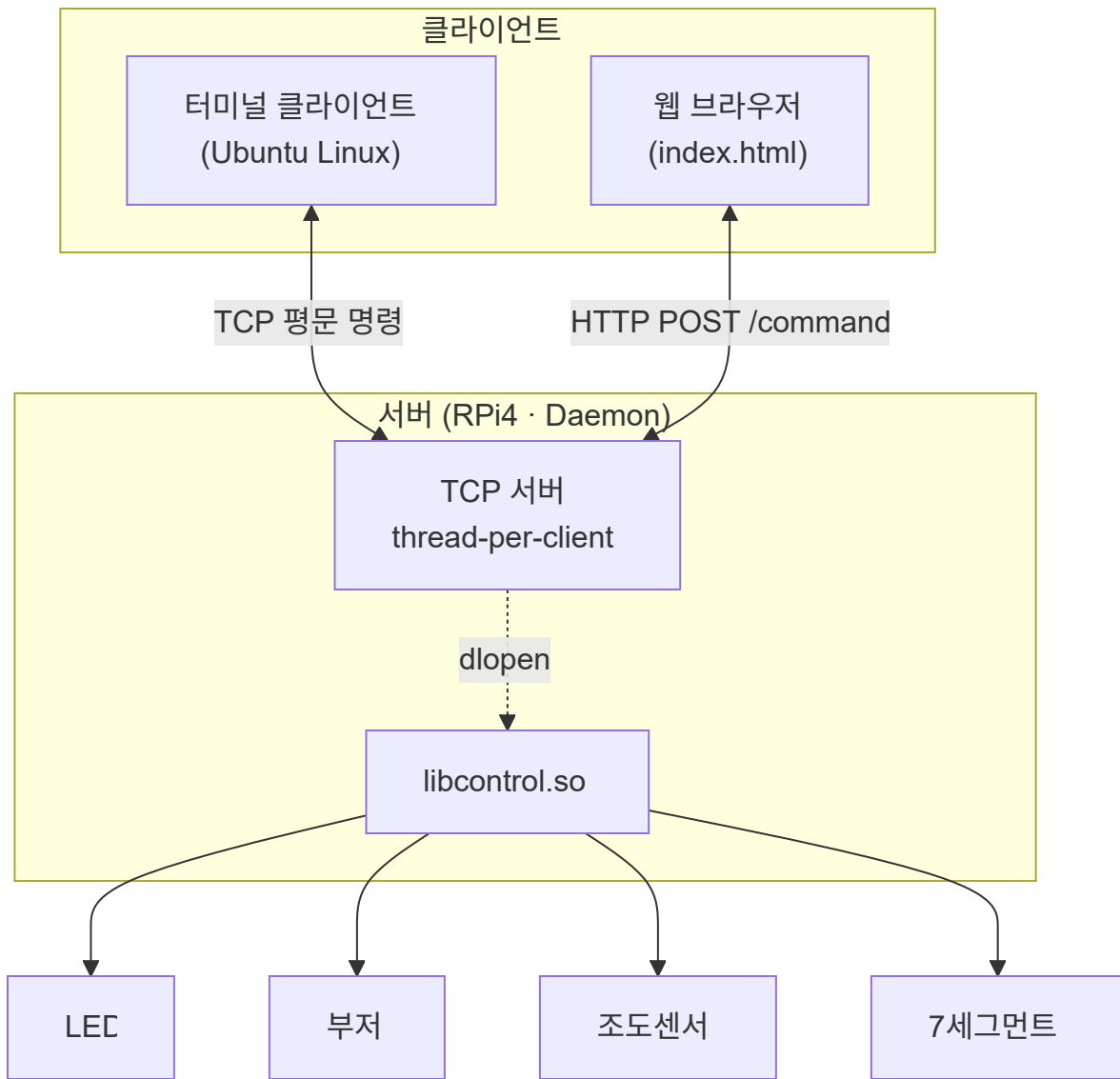
장치	기능
7세그먼트	클라이언트가 전송한 숫자(0~9)를 표시하고 1초마다 -1 감소, 0이 되면 부저 울림

✎ 클라이언트 클라이언트는 터미널 **CLI**와 웹 브라우저 두 가지 형태로 제공된다. 동일한 TCP 서버가 양쪽 모두를 처리한다. HTTP 요청은 서버가 자동 감지하여 CLI 명령과 같은 디스패치 경로로 처리된다.

1.3 개발 환경

구분	내용
서버 H/W	Raspberry Pi 4
서버 O/S	Raspberry Pi OS
클라이언트 O/S	Ubuntu Linux
클라이언트 구조	C 프로그램 (터미널) + HTML/JS (웹)
클라이언트 UI	터미널 메뉴 (CLI) + 웹 UI (브라우저)
개발 언어	C
GPIO 제어	WiringPi
통신	TCP/IP (클라이언트 ↔ 서버), 응답은 HTTP-like 텍스트
연결 처리	thread-per-client (클라이언트당 detached pthread)
빌드 자동화	make (aarch64 cross-compile)

1.4 시스템 구성도



2. 시스템 설계

2.1 전체 아키텍처

서버는 백그라운드에서 상시 동작하는 **데몬 프로세스**이며, 연결 처리는 **클라이언트별 전용 스레드 (thread-per-client)**, 장치 제어는 **장치별 전용 스레드(thread-per-device)**로 수행하는 2계층 구조다. **메인 스레드**는 accept 루프로 연결을 수락하고 클라이언트마다 client_thread 를 생성한다. 각 client_thread 는 자기 소켓에서 명령을 읽어 파싱한 뒤 해당 **장치 스레드 5개**(LED·부저·7세그먼트·CDS·자동조도)로 디스패치한다. 클라이언트가 독립 스레드에서 동작하므로 한 클라이언트의 긴 작업(카운트다운, 부저 재생) 중에도 다른 클라이언트가 다른 장치를 동시에 사용할 수 있고, 연결 해제 시 장치 취소도 자연스럽게 처리된다. 소수의 장기 연결을 다루는 소규모 IoT 제어에 적합한 구조다.

스레드 구성

스레드	역할	생성 시점
메인 스레드 (accept 루프)	연결 수락 후 클라이언트마다 client_thread 생성 · detach	서버 기동 시
client_thread (클라이언트당)	명령 수신·파싱 → devices_dispatch, 연결 해제 시 devices_release	클라이언트 접속 시
led_thread	LED 명령 처리	서버 기동 시
buzzer_thread	BUZZER 명령 처리	서버 기동 시
seg_thread	SEG 명령 처리(카운트다운 포함; 0에서 seg_mutex + buzzer_mutex 동시 획득)	서버 기동 시
cds_thread	CDS READ 명령 처리 (원시값 반환)	서버 기동 시
auto_light_thread	CDS AUTO 명령 처리 (센서 읽기 + 임계값 비교 → LED 자동 제어)	서버 기동 시

각 장치 스레드는 다음을 보유한다: pthread_mutex_t mutex (장치 접근 보호), pthread_cond_t cond (새 명령 신호), int busy (사용 중 여부), int client_fd (응답 대상), Command cmd (처리할 명령).

디스패치 흐름: client_thread 가 명령을 수신·파싱한 뒤 devices_dispatch 를 호출한다 → 장치 mutex lock → busy==1 이면 unlock 후 client_thread 가 503 BUSY 전송, busy==0 이면 cmd / client_fd 저장 · busy=1 · cond signal 후 unlock(즉시 반환, 비블로킹) → 장치 스레드가 깨어나 명령을 실행하고 해당 client_fd 로 직접 응답한 뒤 busy=0 .

장치 자원 조정 (스레드 안전성)

동기화는 두 가지를 구분한다. DeviceThread 의 mutex 는 구조체 필드(busy · cmd · client_fd)를 보호하는 **짧은 잠금**으로, 디스패치 시 검사·갱신 동안만 잡고 즉시 해제한다 (명령 실행 내내 잡지 않는

다 — §2.2 참조). 명령 실행 시간 동안의 "장치 사용 중" 상태는 **busy 플래그**가 담당하며, 같은 장치의 동시 명령은 busy로 차단되어 한 장치는 단일 스레드가 순차 처리한다(라이브러리 자체는 잠금 상태를 갖지 않는다).

물리 핀 접근은 위 직렬화로 대부분 안전하지만, 다음 경우에는 추가 조정이 필요하다.

- **다중 핀 동작**: 7세그먼트 숫자 표시는 여러 세그먼트 핀을 함께 설정하므로 한 단위로 수행한다(seg 는 seg_thread 가 단독 처리하여 다른 스레드가 끼어들지 않는다).
- **카운트다운 0 알림**: seg_thread 가 0에서 부저 핀을 만지므로, 부저 잠금을 짧게 획득해 buzzer_thread 와 충돌하지 않게 한다. 데드락 방지를 위해 획득 순서를 고정한다(예: seg → buzzer).
- **LED 공유**: auto_light_thread (CDS AUTO)와 led_thread 가 같은 LED 핀을 만지므로, LED 잠금을 짧게 획득해 경합을 막는다.

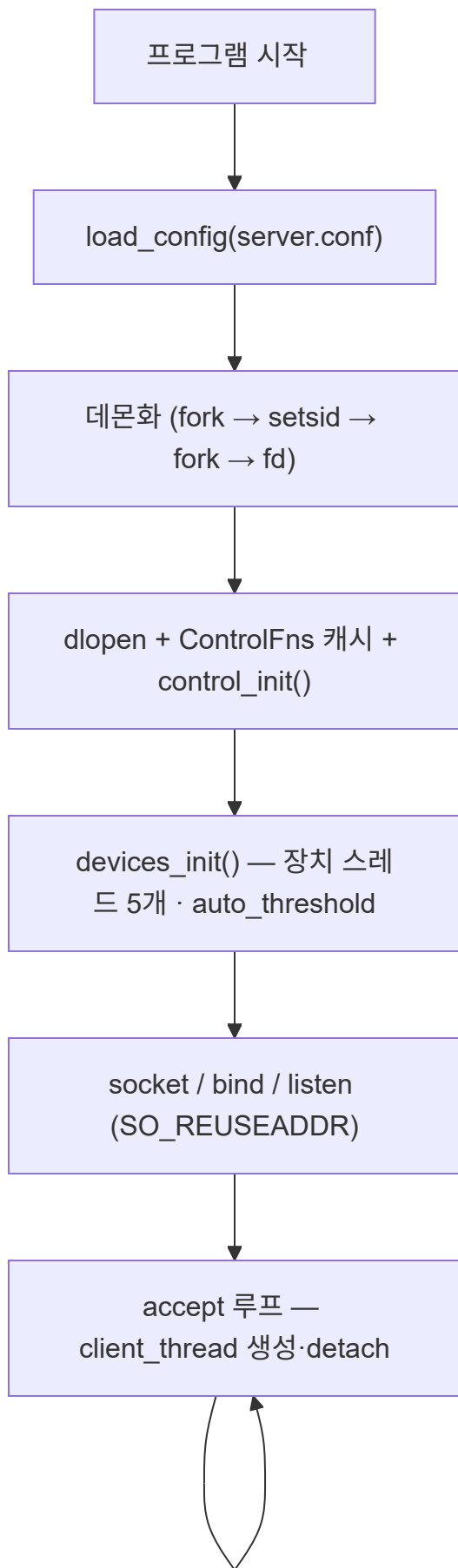
핵심은 전역 단일 뮤텁스를 길게 점유하지 않고, **구조체 보호용 짧은 mutex**(§2.2)와 **핀을 공유하는 지점의 짧은 잠금**만 사용하는 것이다.

초기화 순서

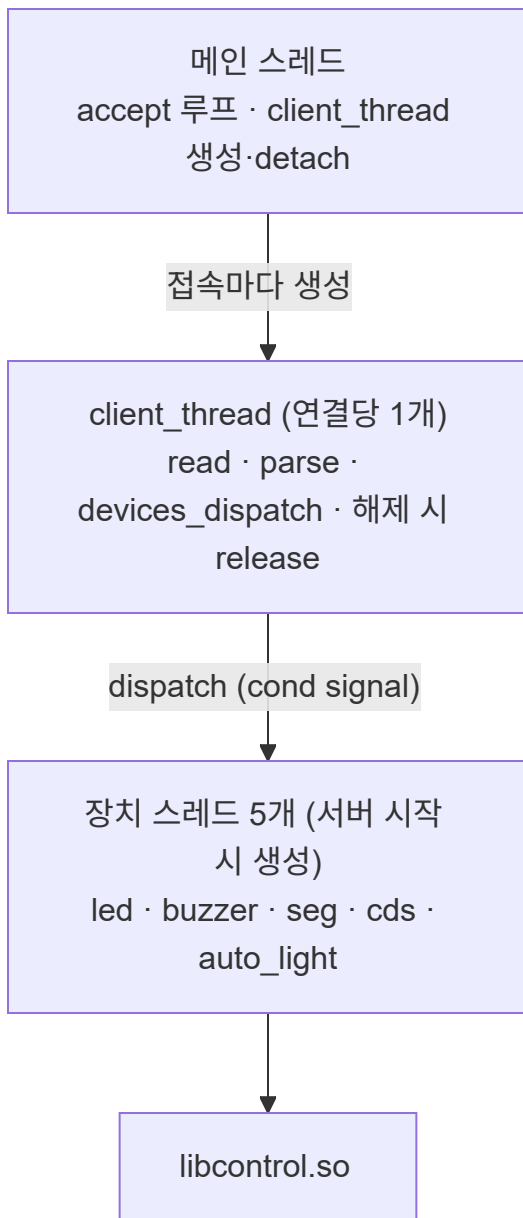
초기화는 server_setup() 이 담당하며 순서는 다음과 같다.

1. load_config("server.conf") — 포트·임계값 로드(파일 없으면 종료)
2. 데몬화 (fork → setsid → fork → fd 리다이렉트)
3. server_setup() : a. dlopen("libcontrol.so") b. 함수 포인터 캐시 (ControlFns) c. control_init() d. devices_init() — **장치 스레드 5개** 생성 및 detach e. auto_threshold 설정 f. 소켓 생성, bind , listen (SO_REUSEADDR)
4. accept 루프 — 접속마다 client_thread 생성·detach

fork() 는 호출 스레드만 복제하므로 장치 스레드는 데몬화(최종 fork) 이후에 생성한다. 또한 장치 스레드는 libcontrol 을 호출하므로 dlopen() · control_init() 이 그 이전에 완료되어야 한다.



런타임 스레드 구조



HTTP 클라이언트 식별

각 `client_thread` 는 첫 수신 데이터의 시작 부분을 검사하여 HTTP 요청(`GET / POST / OPTIONS`)인 경우 `fd`를 **HTTP 레지스트리**에 등록한다. `send_response()` 는 등록된 `fd`에 대해 자동으로 HTTP 헤더를 래핑하여 응답을 전송한다. CLI·웹 양쪽 모두 동일한 디스패치 경로 (`parse_command` → `devices_dispatch`)를 사용한다.

2.2 장치 중재 (busy 플래그)

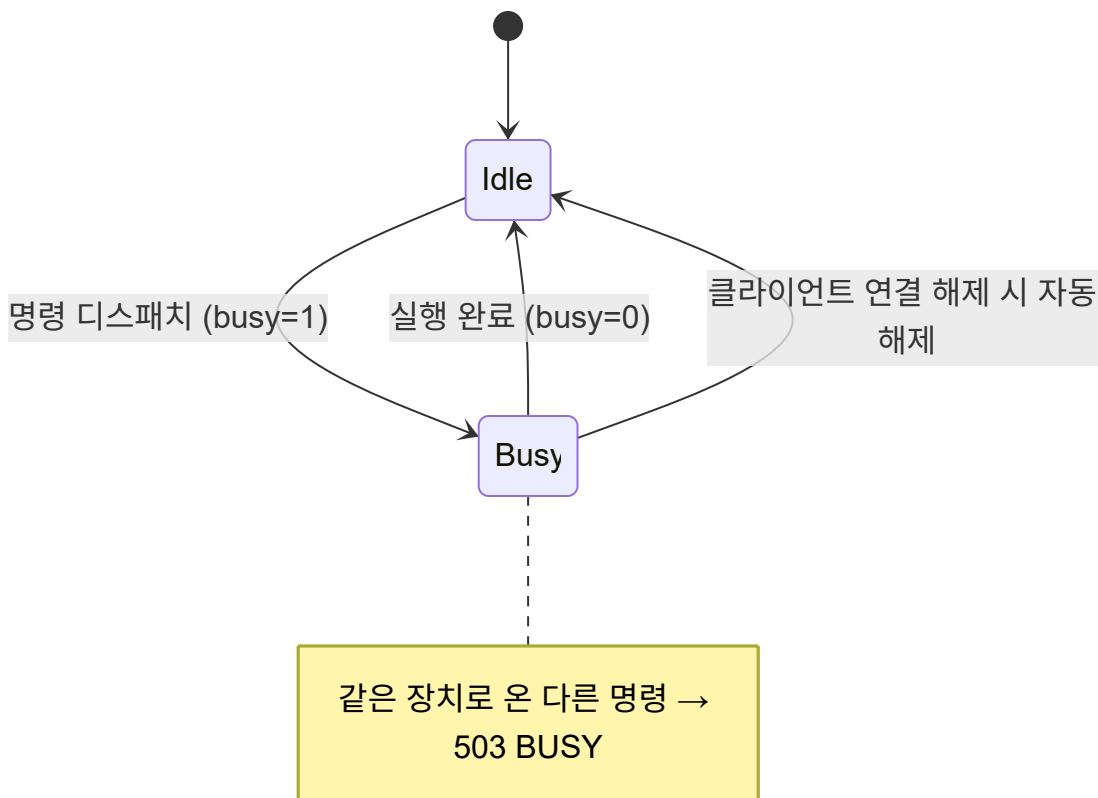
장치 단위 중재는 각 장치 스레드가 보유한 **busy 플래그**로 처리하며, 별도의 중재·인증 로직은 두지 않는다.

규칙

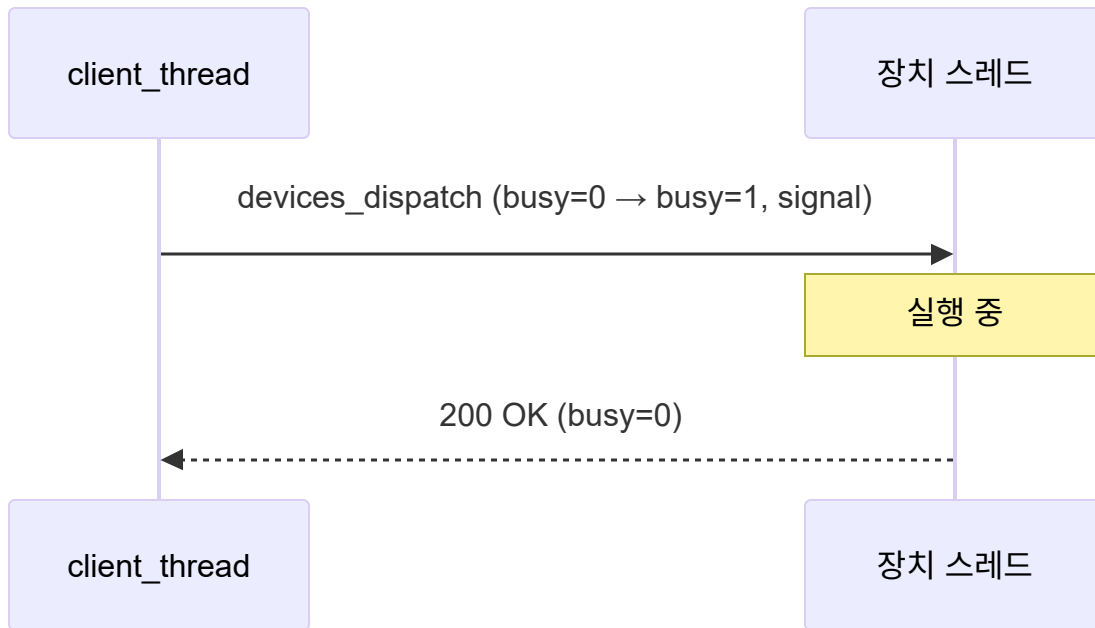
- `client_thread` 가 `devices_dispatch` 로 명령을 넘길 때 장치 `mutex` 를 잡고 `busy` 를 확인한다. `busy==0` 이면 명령을 넘기고 `busy=1` 로 표시, `busy==1` 이면 즉시 `503 BUSY` 를 반환한다.
- 서로 다른 장치는 독립이므로, 서로 다른 클라이언트가 **다른 장치**를 동시에 사용할 수 있다 (예: A는 LED, B는 부저).
- **클라이언트 연결 해제 시**, 해당 `client_thread` 가 종료 직전 `devices_release(fd)` 를 호출해 그 `client_fd` 가 보유한 모든 장치의 `busy` 를 자동 해제한다.

✎ 충돌이 발생하는 지점 LED·부저처럼 순간에 끝나는 명령은 `busy` 점유 시간이 매우 짧아 `503` 이 드물다. `503 BUSY` 는 주로 카운트다운이 진행 중인 7세그먼트에서 발생하며, 그동안에도 LED·부저·CDS는 영향받지 않는다.

장치 busy 상태



디스패치 시퀀스



같은 장치로 다른 클라이언트가 명령을 보내면 `busy==1` 이라 즉시 503 BUSY 를 받는다 (위 busy 상태도 · §2.3 BUSY 흐름 참조).

Mutex와 Busy 플래그의 역할 분리

장치별 동시성 제어는 두 가지 메커니즘을 함께 사용한다. 역할이 명확히 다르므로 둘 중 하나만으로는 충분하지 않다.

메커니즘	보호 대상	점유 시간	목적
Mutex (<code>pthread_mutex_t</code>)	<code>DeviceThread</code> 필드 (<code>busy</code> , <code>cmd</code> , <code>client_fd</code>)	수 μ s	공유 데이터의 동시 접근 방지
Busy 플래그 (<code>int busy</code>)	물리 장치 (LED, 부저 등)	명령 실행 전체 시간	"사용 중"임을 다른 클라이언트에게 알림

Mutex의 역할 — `DeviceThread`의 `busy` · `cmd` · `client_fd`를 두 스레드가 동시에 읽거나 쓰지 못하게 한다. 즉 공유 데이터를 검사·갱신하는 짧은 임계 구역을 보호하며, 검사·갱신을 마치는 즉시 해제된다(명령 실행 내내 잡고 있지 않는다).

```

pthread_mutex_lock(&led_dev.mutex);
/* === 임계 구역 (수십~수백 ns) === */
if (led_dev.busy) {
    pthread_mutex_unlock(&led_dev.mutex);
    return -1;
}
led_dev.busy      = 1;
led_dev.cmd       = *cmd;
  
```

```
led_dev.client_fd = client_fd;
pthread_cond_signal(&led_dev.cond);
pthread_mutex_unlock(&led_dev.mutex);
/* === 즉시 해제 === */
```

Busy 플래그의 역할 — "현재 이 장치가 명령을 처리 중"임을 나타내는 1비트 상태로, 장치 스레드가 명령을 실행하는 전체 시간 동안 1로 유지되다 끝나면 0이 된다. 새 명령이 오면 `busy == 0` 이면 큐잉 (`busy=1` 후 `cond_signal`), `busy == 1` 이면 즉시 503 BUSY로 응답한다.

왜 둘 다 필요한가

- **Mutex만 쓰면** — 카운트다운 같은 긴 명령이 락을 ~10초간 점유하면, 그 사이 들어온 SEG CLEAR는 `pthread_mutex_lock`에서 수 초간 블로킹된 뒤에야 응답한다(head-of-line blocking). 사용자에게는 "눌렀는데 몇 초간 무반응"으로 보인다.
- **Busy 플래그만 쓰면** — 두 스레드가 `busy`를 동시에 읽어 둘 다 0으로 보고 둘 다 1로 쓰는 race condition이 생겨, 같은 장치를 동시에 점유하고 `cmd/client_fd` 갱신이 깨진다.
- **협력** — `busy` 검사·갱신이 원자적이어야 의미를 갖고, 그 원자성을 mutex가 제공한다. 반대로 mutex를 짧게만 점유하려면 "일하는 중"이라는 별도 상태가 필요한데 그것이 `busy`다.

동작 시나리오

- **짧은 명령 (LED ON)** — `softPwmWrite` 한 번으로 끝나 `busy=1`이 수 μ s만 유지되므로 동시 LED OFF가 503 BUSY를 받을 확률은 극히 낮다.
- **긴 명령 (SEG COUNTDOWN 9)** — 약 10초간 `seg_dev.busy=1`이 유지되어, 그 사이 어떤 SEG 명령도 즉시 503 BUSY를 받는다(빠른 피드백 → 재시도/안내 가능).
- **서로 다른 장치 동시 사용** — `busy`는 장치별로 존재하므로, SEG 사용 중에도 다른 클라이언트가 LED·BUZZER·CDS를 자유롭게 쓸 수 있다.

 **비유: 진료실 Mutex는 안내데스크의 키보드(일정 입력 시 잠깐 점유), Busy 플래그는 진료실 문의 "진료 중" 표지판(진료 내내 걸려 있음)이다. 표지판이 없으면 다음 환자가 그냥 들어가고, 키보드 규칙이 없으면 두 직원이 예약장을 동시에 고쳐 엉망이 된다. 둘 다 있어야 올바르다.**

질문	Mutex	Busy 플래그
무엇을 보호	구조체 필드 (메모리)	물리 장치 (하드웨어)
점유 시간	수 μ s	명령 실행 시간 (수 ms ~ 수 s)
경합 시	짧게 대기 후 진입	즉시 503 BUSY 반환
없으면	데이터 손상 (race condition)	Head-of-line blocking

이 **역할 분리** — mutex는 데이터 정합성을, busy 플래그는 응답성을 보장 — 가 장치별 스레드 모델의 핵심이다.

2.3 통신 프로토콜

클라이언트와 서버는 **줄 단위 텍스트 프로토콜**로 통신한다. 요청은 줄바꿈(\n)으로 구분되고 공백으로 나뉜 토큰으로 구성된다(ASCII). 응답은 **HTTP-like 형식**(상태 줄 + 선택적 본문)으로 통일하여, 추후 웹 클라이언트 추가를 용이하게 한다. 메뉴 기반 터미널 클라이언트가 선택한 명령을 TCP로 전송하고 서버 응답을 출력한다.

서버 설정 파일 (server.conf)

서버는 시작 시 `server.conf` 를 읽는다. `port` 는 수신 포트, `threshold` 는 CDS AUTO 명령에서 LED 제어 판단 기준값이다. 파일이 없으면 서버가 종료된다.

```
port 8080
threshold 500
```

명령어 (클라이언트 → 서버)

명령	설명	응답(예)
LED ON / LED OFF	LED 켜기 / 끄기	200 OK
LED BRIGHT <0/1/2>	밝기 3단계(0=최저, 1=중간, 2=최대)	200 OK
BUZZER ON / BUZZER OFF	부저 톤 재생 / 정지	200 OK
SEG DISPLAY <0-9>	숫자 표시	200 OK
SEG CLEAR	7세그먼트 끄기	200 OK
SEG COUNTDOWN <0-9>	해당 숫자부터 카운트다운(0에서 부저)	200 OK
CDS READ	조도센서 원시값 조회	200 OK + value=<n>
CDS AUTO	센서 읽기 + 임계값 비교 → LED 자동 제어	200 OK + value=<n> + led=ON/OFF

서버 응답 (HTTP-like)

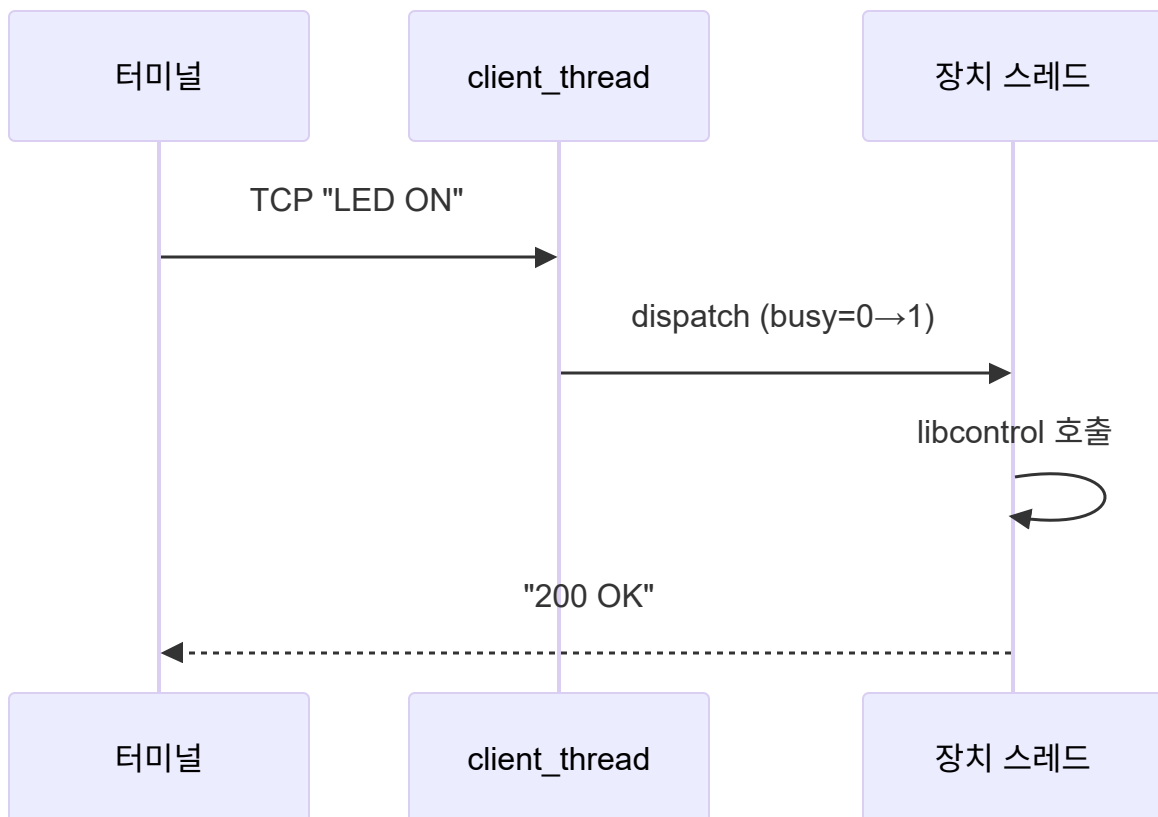
모든 응답은 아래 형식 중 하나를 따른다.

상황	응답
연결 수락	200 OK\n
명령 성공	200 OK\n
장치 사용 중	503 BUSY\n
잘못된 명령	400 BAD REQUEST\n
CDS READ 성공	200 OK\nvalue=<n>\n
CDS AUTO 성공	200 OK\nvalue=<n>\nled=ON/OFF\n

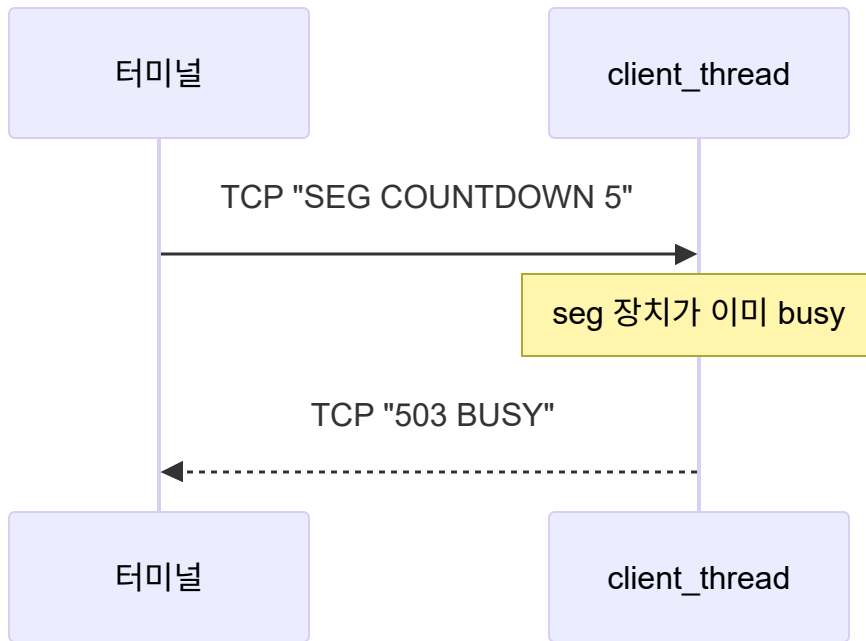
동작 규칙

- LED BRIGHT <0/1/2> 명령은 LED를 켜 상태로 두고 밝기 단계를 설정한다.
- 같은 장치가 사용 중이면 명령은 503 BUSY 를 받는다(2.2 참조). 클라이언트는 잠시 후 재시도한다.
- CDS AUTO 의 밝음/어두움 판단은 서버가 원시값을 server.conf 의 threshold 와 비교해 수행한다.

기본 명령 흐름



사용 중(BUSY) 흐름



HTTP 프로토콜 (웹 클라이언트)

웹 브라우저는 원시 TCP를 직접 열 수 없으므로 HTTP로 요청한다. 서버는 같은 포트에서 HTTP와 CLI를 모두 받는다.

요청	처리
GET / 또는 GET /index.html	웹 UI HTML 반환
POST /command (body: 명령 텍스트)	본문에서 명령 추출 → CLI와 동일한 경로로 처리
POST /disconnect	탭 닫힘 알림 — 해당 fd가 보유한 장치 정리, 연결 종료
GET /favicon.ico	204 No Content
OPTIONS *	CORS preflight

요청 파싱 (http_parse) — client_thread는 수신 버퍼의 첫 토큰이 GET / POST / OPTIONS 이면 HTTP로 판별한다. http_parse()는 (1) 요청 라인(<METHOD> <path> HTTP/1.1)에서 메서드와 경로를 분리하고, (2) 헤더와 본문의 경계인 **빈 줄(\r\n\r\n)**을 찾아 그 뒤를 본문으로 취한다(필요 시 Content-Length로 길이 확인). POST /command의 경우 **본문 문자열이 곧 장치 명령**이며, 이를 CLI한 줄과 똑같이 parse_command에 넘긴다 — 즉 두 경로가 여기서 합류한다. 경로가 / 이면 index.html, /favicon.ico 이면 204, OPTIONS 이면 CORS preflight 응답을 반환한다. HTTP로 판별된 fd는 **HTTP 레지스트리**에 등록되어, 이후 send_response()가 상태 줄·CORS·Content-Type / Content-Length 헤더를 본문(CLI 응답 그대로)에 씌워 전송한다.

HTTP 응답은 CLI 응답(200 OK\n, 503 BUSY\n 등)에 HTTP 헤더를 래핑하여 전송한다. 응답 본문은 CLI와 동일한 형식이므로 JavaScript에서 r.text()로 받아 그대로 표시할 수 있다.

2.4 공유 라이브러리 구조 (libcontrol)

모든 장치 제어 로직은 **단일 공유 라이브러리 libcontrol.so** 로 빌드한다. 서버는 시작 시 이 라이브러리를 `dlopen()` 으로 로드하고, `dlsym()` 으로 각 함수의 포인터를 얻어 호출한다. 이렇게 하면 장치 제어 코드를 수정한 뒤 `libcontrol.so` 파일만 교체(업로드)하고 서버를 재시작하면 새 동작이 반영되므로, 서버 본체를 다시 빌드할 필요가 없다.

공개 인터페이스 (control.h)

라이브러리는 **상태 없는 제어 원시 함수(primitive)** 만 제공한다. 카운트다운·자동 연동 같은 **시간/스레드가 얹힌 오케스트레이션**은 서버가 담당한다(라이브러리를 단순·교체 가능하게 유지하기 위함).

```
/* 초기화 / 정리 */
int control_init(void);          /* wiringPiSetup + 핀 모드 설정, 성공 0 */
void control_cleanup(void);      /* 핀 초기 상태로 복원 */

/* LED */
void led_on(void);
void led_off(void);
void led_brightness(int level); /* 0=LOW, 1=MID, 2=HIGH (softPwm) */

/* 부저 (Buzzer) */
void buzzer_on(void);            /* 부저 톤 켜기 (기본 주파수) */
void buzzer_off(void);           /* 부저 톤 끄기 */
void buzzer_tone(int freq);      /* 단일 주파수 톤 - buzzer_thread의 음 시퀀스에
서 사용 */

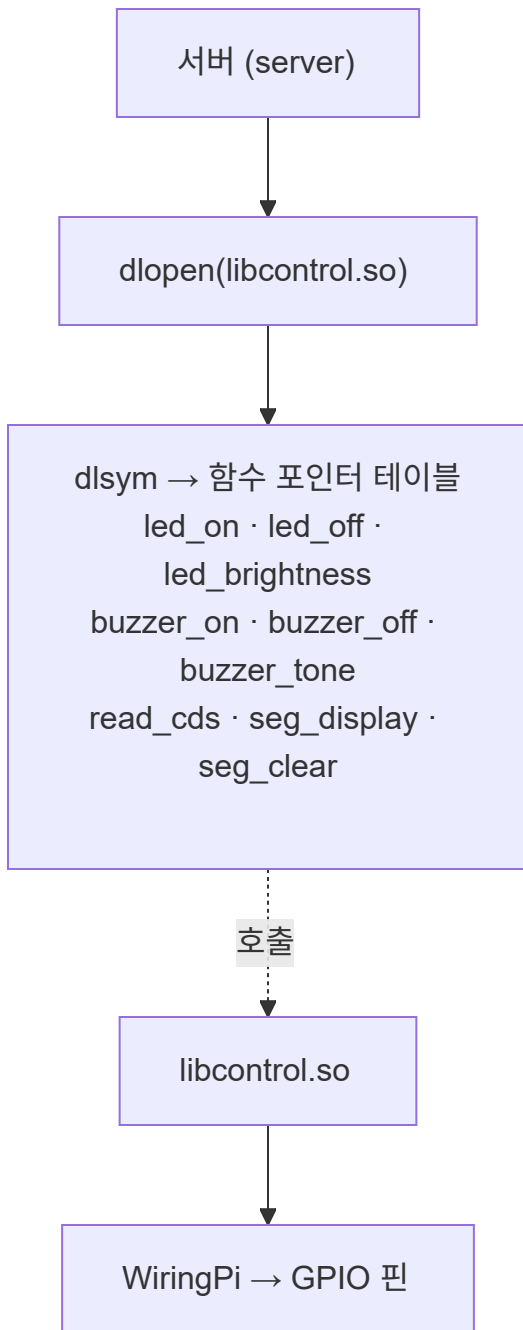
/* 조도센서 (I2C 아날로그 CDS, 0x48) */
int read_cds(void);              /* I2C CDS 센서 원시값 반환 (0~255) */

/* 7세그먼트 (단일 자리) */
void seg_display(int digit);     /* 0~9 표시 */
void seg_clear(void);

/* internal - control_init()에서만 호출 */
void led_init(void);
void buzzer_init(void);
void cds_init(void);
void seg_init(void);
```

부저의 음 시퀀스(Happy Birthday) 재생은 **서버의 buzzer_thread** 가 담당하여 라이브러리는 상태 없는 원칙을 유지한다. `buzzer_thread` 는 melody 배열을 순회하며 음마다 `fns.buzzer_tone(freq)` → `usleep` → `fns.buzzer_off()` 를 호출하고, 음 사이마다 `cancel` 플래그를 확인하여 재생 중에도 즉시 중단할 수 있다.

로딩 구조



2.5 디렉터리 / 모듈 구조

```

project/
├── server/
│   ├── server.c           # main(): daemonize → server_setup → accept 루프
│   ├── daemon.c / .h      # 데몬화 (double-fork + 로그 리다이렉트)
│   ├── client_handler.c / .h # 연결당 client_thread 함수 (HTTP/CLI 감지·디스패치)
│   ├── setup.c / .h       # dlopen, control_init, devices_init, 소켓 설정
│   ├── config.c / .h      # server.conf 파싱
│   ├── protocol.c / .h    # 명령 파싱
│   ├── http.c / .h        # HTTP 요청 파싱, HTML 제공
│   └── devices.c / .h      # 장치 스레드, 디스패치, HTTP fd 레지스트리,
  
```

```

send_response
|   └─ led_thread.c           # LED 장치 스레드
|   └─ buzzer_thread.c       # 부저 장치 스레드 (melody 시퀀스 + cancel)
|   └─ seg_thread.c          # 7세그먼트 장치 스레드 (카운트다운 포함)
|   └─ cds_thread.c          # CDS 장치 스레드 (CDS READ)
|   └─ auto_light_thread.c    # CDS AUTO 장치 스레드
|   └─ index.html            # 웹 클라이언트 UI
|   └─ server.conf           # 설정 파일 (port, threshold)
└─ libcontrol/
   └─ control.c              # control_init / control_cleanup (장치 공통 초기화·정
리)
   └─ led.c                  # LED 제어 (softPwm on/off/밝기)
   └─ buzzer.c               # 부저 제어 (톤 on/off, buzzer_tone)
   └─ cds.c                  # 조도센서 I2C CDS 원시값 읽기
   └─ seg.c                  # 7세그먼트 숫자 표시
   └─ control.h              # 공개 인터페이스
└─ client/
   └─ client.c               # 메뉴 기반 터미널 클라이언트
└─ Makefile
└─ README.md

```


3. 세부 구현 내용

3.1 서버 데몬화 (Daemon)

데몬화는 `daemon.c` 의 `daemonize(log_path)` 함수가 담당하며, `main()` 에서 `load_config` 직후 호출된다(설정 파일 로드 실패 시 오류 메시지를 터미널에서 볼 수 있도록 데몬화 전에 처리). 표준 절차를 따른다.

1. `fork()` → 부모 프로세스 종료 (셸이 자식을 백그라운드로 인식)
2. `setsid()` → 새 세션 리더가 되어 제어 터미널과 분리
3. 두 번째 `fork()` → 세션 리더 포기 (제어 터미널 재획득 방지)
4. `umask(0)` → 파일 모드 정리
5. `stdin` 을 `/dev/null` 로 리다이렉트
6. `stdout` / `stderr` 를 `server.log` (append 모드)로 리다이렉트
7. `stdout` / `stderr` 라인 버퍼링 활성화 (로그가 즉시 기록되도록)

작업 디렉터리(`chdir`)는 변경하지 않는다 — `server.conf · index.html` 같은 상대 경로 파일 접근을 유지하기 위함이다. 부모 종료 전 자식 PID를 출력하여, 종료 시 `kill <PID>` 또는 `pkill -x server` 로 정지할 수 있다.

`fork()` 는 호출 스레드만 복제하므로 **장치 스레드는 데몬화(최종 `fork`) 이후에 생성한다** (2.1 참조).

3.2 멀티스레드 장치 제어

서버 스레드 구성은 2.1과 같다. 구현 관점의 책임은 다음과 같다.

- **메인 스레드 (accept 루프)** — `server.c` 의 `main()` . `accept()` 만 호출하고, 새 연결마다 `malloc()` 으로 fd 복사본을 만들어 `pthread_create(client_thread, ...)` 로 클라이언트 스레드를 생성·detach한다. `malloc` 을 쓰는 이유는 스택 변수를 직접 전달하면 다음 `accept` 루프 이전에 값이 덮어써지는 경쟁 조건이 발생하기 때문이다.
- **client_thread (연결당 1개)** — `client_handler.c` . `read()` 로 데이터를 수신하고, 첫 바이트로 HTTP/CLI 여부를 판단한다. HTTP면 `http_parse()` 로 본문에서 명령을 추출하고 HTTP 레지스트리에 fd를 등록한다. **양쪽 모두 같은 `parse_command()` → `devices_dispatch()` 경로로 처리된다.** 연결 해제 시 `devices_release(fd)` 로 보유 장치를 자동 해제하고 종료한다.
- **장치 스레드 (led/buzzer/seg/cds/auto_light)**: 서버 시작 시 생성. `cond` 로 새 명령을 대기하다가 깨어나면 `libcontrol` 을 호출해 명령을 실행하고 `client_fd` 로 응답한 뒤 `busy=0` .
- **seg_thread 의 카운트다운**: `SEG COUNTDOWN <n>` 처리 시 1초 간격으로 `seg_display, 0`에서 `buzzer_on` (이때 `seg_mutex + buzzer_mutex` 동시 획득). `SEG CLEAR` 또는 새 카운트다운으로 취소·교체.
- **auto_light_thread**: `CDS AUTO` 처리 시 센서 원시값을 읽어 `threshold` 와 비교 → LED를 자동 on/off하고 `value · led` 상태를 응답한다(LED 접근 시 `led_mutex` 획득).

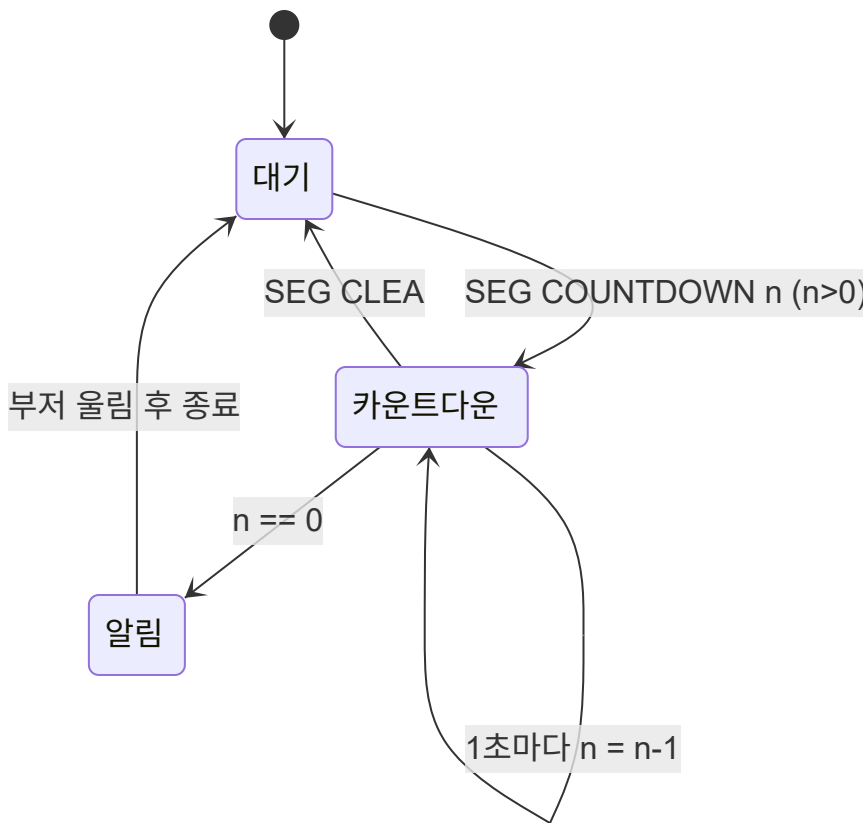
각 클라이언트가 독립 스레드이므로 한 클라이언트의 긴 작업 중에도 다른 클라이언트는 정상 동작한다. 같은 장치에 동시에 온 명령은 `busy` 확인으로 503 BUSY 처리된다(2.2 참조). 연결 급증 시 스레드 풀을 도입해 스레드 수를 제한할 수 있다.

3.3 장치별 구현

 **핀 매핑 (실제 배선 기준, WiringPi 번호) 아래는 실제 확정된 배선이다.**

장치	핀	비고
LED	1	softPwm (PWM_RANGE=100)
부저	6	softTone, passive buzzer
조도센서	I2C 0x48	wiringPiI2CSetup
7세그먼트 a	27	common anode (ON=LOW)
7세그먼트 b	28	
7세그먼트 c	29	
7세그먼트 d	25	
7세그먼트 e	24	
7세그먼트 f	23	
7세그먼트 g	22	

7세그먼트 카운트다운 상태



3.4 시그널 처리

요구사항: 클라이언트는 실행 중 강제 종료되지 않고 **SIGINT**에서만 종료한다.

- **클라이언트**: sigaction 으로 SIGINT 핸들러 등록 → 소켓 정리·메시지 출력 후 정상 종료.
SIGQUIT, SIGHUP, SIGTERM, SIGPIPE, SIGTSTP 등 나머지 캐치 가능한 시그널은 모두 무시 (SIG_IGN)하여 실행 중 비정상 종료를 방지한다. (SIGKILL · SIGSTOP 은 커널이 강제하므로 처리 불가)
- **서버(데몬)**: SIGHUP 무시 또는 설정 재로딩 용도, SIGTERM 수신 시 스레드 정리·control_cleanup 후 종료.

3.5 빌드 자동화

make 만 사용한다. 서버·라이브러리는 **aarch64** 크로스 컴파일, 클라이언트는 Ubuntu 네이티브 빌드한다.

대상	빌드 방식
libcontrol.so	aarch64-linux-gnu-gcc -fPIC -shared -lwiringPi -lpthread
server	aarch64-linux-gnu-gcc -ldl -lpthread (cross-compile)
client	gcc -lpthread (Ubuntu 네이티브)

make 타겟: all, libcontrol, server, client, 그리고 배포용 send (server + index.html 을 Pi로 scp), send_server (서버 바이너리만), send_lib (라이브러리만), clean. 빌드/실행 방법은 README에 정리한다(제출물 요구사항).

3.6 클라이언트 (메뉴 기반 터미널)

클라이언트는 계층적 터미널 메뉴를 통해 장치를 제어한다. 메뉴에서 명령을 선택하면 해당 TCP 명령을 전송하고 서버 응답을 즉시 출력한다. 추후 웹 클라이언트로 확장할 수 있도록 프로토콜은 HTTP-like 형식을 사용한다.

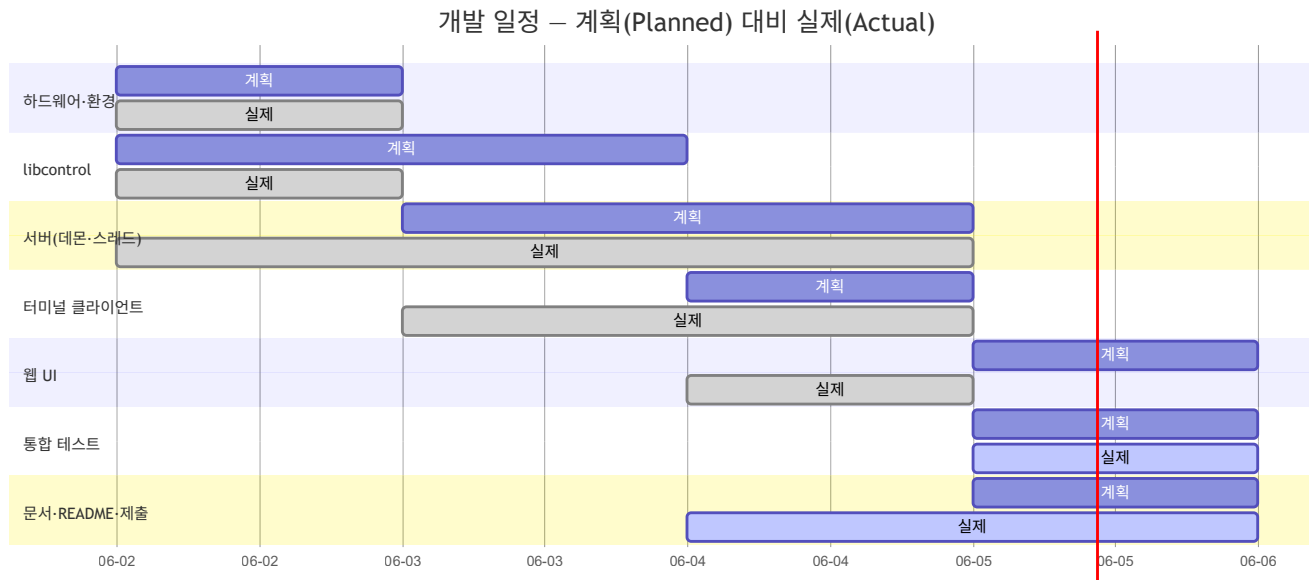
```
Main Menu
├─ 1. Manual Mode
|   ├── 1. LED      → ON / OFF / BRIGHT (0-2)
|   ├── 2. BUZZER   → ON / OFF
|   ├── 3. SEG      → DISPLAY (0-9) / CLEAR / COUNTDOWN (0-9)
|   ├── 4. CDS      → READ
|   └─ 5. Back
├─ 2. Automatic Mode → CDS AUTO (센서 읽기 + LED 자동 제어)
└─ 3. Quit
```

시그널 처리는 3.4와 같다(SIGINT 만 정상 종료, 나머지 무시).

BUZZER ON 선택 시 터미널을 raw 모드(termios 의 ICANON / ECHO 해제)로 전환하고 poll() 로 stdin과 소켓을 동시에 감시한다. 노래가 끝나기 전에 아무 키나 누르면 즉시 BUZZER OFF 명령이 전송되어 노래가 중단된다. 메뉴 복귀 시 터미널 상태를 복원한다.

4. 개발 일정

제출 기한은 6월 5일 15시이다. 아래 차트는 **계획(Planned)** 대비 **실제(Actual)**를 나란히 두어 진행을 추적하기 위한 것이다. 실제 막대는 작업이 진행되는 대로 시작·기간을 실제 값으로 수정하고, 완료 시 막대 앞에 `done`, 진행 중이면 `active`, 를 붙인다. 차트에는 오늘 날짜를 나타내는 세로선이 자동 표시되어 비교에 도움이 된다.



5. 추가 기능

5.1 웹 UI

웹 브라우저 기반 제어 UI를 추가로 구현하였다. **별도의 웹 서버를 두지 않고, 같은 TCP 서버가 HTTP와 CLI를 모두 처리한다.**

동작 방식

- `client_thread` 가 수신 데이터의 첫 바이트로 HTTP 여부를 판단한다(`GET / POST / OPTIONS`).
- `http_parse()` 로 HTTP 요청을 분석 → `POST /command` 의 본문에서 명령 문자열을 추출한다.
- 추출된 명령은 CLI 명령과 **완전히 동일한 경로**(`parse_command` → `devices_dispatch`)로 처리된다.
- 응답 시 `send_response()` 가 HTTP 레지스트리에 등록된 fd에 대해 자동으로 HTTP 헤더를 래핑한다.
- `index.html` 은 서버 작업 디렉터리에서 `GET /` 요청 시 제공된다.
- 탭 닫힘 시 `navigator.sendBeacon('/disconnect')` 로 해당 fd 소유 장치를 정리한다.

UI 구성: LED on/off·밝기 3단계, 부저 노래 재생·정지, 7세그 표시·CLEAR·카운트다운, CDS READ·AUTO, 응답 로그.

제약 사항: 브라우저 연결 풀의 동작 특성상 fd가 요청마다 달라질 수 있어 "같은 사용자"를 fd로 식별할 수 없다. 신뢰된 LAN 환경에서는 영향이 없으나 실제 운영에서는 세션 토큰이 필요하다.

5.2 다중 클라이언트 접속 제어

여러 클라이언트(터미널·웹 혼합)가 **동시에 접속**하여 장치를 제어할 수 있도록 구현하였다. 연결 처리와 장치 중재가 함께 이를 보장한다.

- **동시 접속:** thread-per-client 구조로 접속마다 독립 `client_thread` 를 생성하므로, 한 클라이언트의 긴 작업(카운트다운·부저 재생) 중에도 다른 클라이언트가 차단되지 않는다(2.1 참조).
- **장치 단위 중재:** 장치별 `busy` 플래그로 같은 장치에 대한 동시 명령을 조정한다. 사용 중인 장치로 명령이 오면 대기 없이 즉시 503 BUSY 로 응답하여 **빠른 피드백**을 준다(2.2 참조).
- **독립 장치 병행:** `busy` 는 장치별로 존재하므로 서로 다른 장치(예: A는 LED, B는 부저)는 여러 클라이언트가 동시에 사용할 수 있다.
- **자동 정리:** 클라이언트 연결이 끊기면 해당 `client_thread` 가 `devices_release(fd)` 로 그 클라이언트가 점유하던 장치를 자동 해제하여, 비정상 종료 시에도 장치가 잠긴 채 남지 않는다.

별도의 우선순위·대기 큐는 두지 않으며(단순한 선착순 + 즉시 거절 정책), 이는 신뢰된 LAN·소수 클라이언트 환경에 맞춘 선택이다(확장 방향은 §6 참조).

6. 문제점 및 보완 사항

본 설계의 알려진 한계와 향후 보완 방향을 정리한다. 일부는 과제 범위·환경(신뢰된 LAN, 장치 4개)에 맞춘 **의도적 선택**이며, 실제 운영 수준에서는 아래와 같이 개선할 수 있다.

항목	한계 / 문제점	보완 방향
시스템 통합	직접 구현한 데모화 — PID 파일·자동 재시작 없음	<code>systemd</code> 서비스 단위로 전환 — <code>journald</code> 로깅, 자동 재시작, 의존성 관리
타이머	카운트다운을 <code>seg_thread</code> 의 <code>sleep</code> 기반으로 처리	정밀 타이밍이 필요하면 타이머 추상화 도입
라이브러리 교체	실행 중 <code>.so</code> 핫스왑은 안전하지 않음	파일 교체 후 서버 재시작(현재 방식). 무중단이 필요하면 버전드 로딩·프로세스 교체
보안	인증·권한 부재 — LAN 내 누구나 장치 제어 가능	필요 시 인증/권한, TLS, 접근 제한 도입
HTTP 클라이언트 식별	브라우저 연결 풀로 인해 같은 탭의 요청도 다른 fd를 사용할 수 있음	세션 토큰(쿠키 등)으로 식별. 신뢰된 LAN에서는 불필요
스레드 자원	클라이언트당 <code>pthread</code> 1개 — 수천 동시 연결 시 비효율	<code>epoll</code> + 워커 풀로 전환 가능. 현재 시나리오(소수 클라이언트, 장기 연결)에는 적합
플러그인 계층	장치 4개에는 다소 과한 추상화	과제 요구(공유 라이브러리)·확장성 시연 목적의 의도적 선택. 단순화 시 정적 링크로 대체 가능

7. 결론

본 프로젝트는 Raspberry Pi 4를 서버로, Ubuntu를 클라이언트로 하는 **TCP 기반 원격 장치 제어 시스템**을 구현하였다. 서버는 데몬으로 동작하며, LED·부저·조도센서(I2C CDS)·7세그먼트 네 가지 장치를 단일 공유 라이브러리 `libcontrol.so`로 추상화하고 `dlopen`으로 동적 로드한다. 라이브러리는 상태 없는 제어 원시 함수만 제공하고, 카운트다운·부저 노래 재생 같은 시간·스레드가 얹힌 동작은 서버가 오케스트레이션하여 라이브러리를 단순하고 교체 가능하게 유지한다.

동시성은 **thread-per-client**(연결마다 독립 스레드)와 **장치별 스레드**(thread-per-device)의 2계층으로 처리한다. 장치별 `busy` 플래그가 같은 장치에 대한 충돌을 즉시 503 BUSY로 거절해 빠른 피드백을 주고, 구조체 보호용 `mutex`가 데이터 정합성을 보장한다. 이 구조 위에서 터미널 CLI와 웹 브라우저가 **같은 TCP 서버**를 통해 동시에 장치를 제어하며, 클라이언트 연결이 끊기면 점유 장치가 자동 해제된다.

결과적으로 과제 요구사항 — 네 장치 제어, 공유 라이브러리, 시그널 처리, 데몬화, 멀티스레드, 빌드 자동화(make) — 를 충족하고, 추가로 **웹 UI**와 **다중 클라이언트 접속 제어**를 구현하였다.